

Boot Spring Boot

스프링 부트 이용설명서

Written by honeymon

Boot Spring Boot

부트 스프링 부트

Jake Jiheon Kim

Version 0.0.2, 2016-10-10

Table of Contents

저자 소개	2
들어가며.....	4
책에서 다루는 내용	4
책을 따라하기 위해 필요항목.....	5
대상독자.....	5
관례(Convention)	5
예제	6
1. 시작하기	8
1.1. 스프링 부트 소개.....	8
1.1.1. 단독실행 가능한 스프링 애플리케이션 생성	8
1.1.2. 내장 컨테이너로 톰캣, 제티 혹은 언더토우 중에서 선택가능(WAR 배포 필요없음).....	8
1.1.3. 스타터 'starter' POM 을 제공하여 간결한 의존성 구성 지원.....	8
1.1.4. 스프링에 대한 {auto-cogfiguration} 제공	9
1.1.5. 제품출시 후 운영에 필요한 상태, 건강 점검과 외부구성 등 다양한 기능 제공	9
1.1.6. 구성을 위한 XML 코드를 필요로 하지 않으며 작성하지 않아도 됨	9
1.2. 시스템 요구사항	9
1.3. 스프링 부트 설치	10
1.4. 스프링 부트 애플리케이션 개발 시작.....	11
1.4.1. 프로젝트 생성	11
1.4.2. 의존성 추가	13
1.4.3. 예제: ExampleController 작성.....	13
1.4.4. 예제: ExampleController 실행하기	14
1.4.5. 실행가능한 JAR 만들기.....	14
1.5. 이어질 이야기.....	17
2. 첫 스프링 부트 애플리케이션 살펴보기.....	18
2.1. 빌드 시스템.....	18
2.1.1. 의존성 관리	18
2.1.2. 스타터	18
2.1.3. 메이븐	18
2.1.4. 그레이들	18
2.1.5. 래퍼	18
2.2. 코드 구조	18
2.2.1. "기본 패키지" 이용	18
2.2.2. 애플리케이션 메인 클래스 위치	20
2.3. 구성(Configuration) 클래스.....	22
2.3.1. 추가적인 구성클래스 불러오기	22
2.3.2. XML 구성 불러오기	22
2.4. 자동구성(Auto-configuration)	22
2.4.1. 자동구성 대체하기.....	22
2.4.2. 특정 자동구성 비활성화하기.....	22
2.5. 스프링 빈과 의존성 주입	22
2.6. @SpringBootApplication 애노테이션 이용	22
2.7. 애플리케이션 실행하기	23
2.7.1. IDE 에서 실행하기.....	23
2.7.2. 패키징된 애플리케이션 실행하기	23

2.7.3. 메이븐 플러그인으로 실행하기	23
2.7.4. 그레이들 플러그인으로 실행하기	23
2.7.5. 핫 스와핑	23
2.8. 스프링 부트 개발자도구	23
2.8.1. 기본 속성들	23
2.8.2. 자동 재시작	23
2.8.3. 전역 설정	23
2.8.4. 원격 애플리케이션	23
2.9. 출시를 위한 애플리케이션 패키징	23
2.10. 이어질 이야기	23
3. 스프링 부트 기능	24
3.1. 스프링애플리케이션(SpringApplication)	24
3.1.1. 배너 재정의	24
3.1.2. SpringApplication 재정의	24
3.1.3. 유연한(Fuent) 빌더 API	24
3.1.4. 애플리케이션 이벤트 및 리스너	24
3.1.5. 웹 환경	24
3.1.6. 애플리케이션 실행 인자에 접근	24
3.1.7. ApplicationRunner 혹은 CommandLineRunner 사용하기	24
3.1.8. 애플리케이션 종료	24
3.1.9. 관리자 기능	24
3.2. 외부 구성	24
3.2.1. 랜덤값 구성	24
3.2.2. 명령줄 속성 접근	24
3.2.3. 애플리케이션 속성 파일	24
3.2.4. 프로파일 정의 속성	24
3.2.5. 속성 내 치환자(placeholder)	24
3.2.6. properties 를 대신하여 YAML 사용하기	24
3.2.7. 보장형 구성 속성	25
3.3. 프로파일	25
3.3.1. 활성화할 프로파일 추가하기	25
3.3.2. 프로파일을 프로그래밍적으로 설정	25
3.3.3. 프로파일 정의 구성 파일	25
3.4. 로깅	25
3.4.1. 로그 형식	25
3.4.2. 콘솔 출력	25
3.4.3. 파일 출력	25
3.4.4. 로그 레벨	25
3.4.5. 로그 구성 재정의	25
3.4.6. 로그백 확장	25
3.5. 웹애플리케이션 개발	25
3.5.1. 'Spring Web MVC framework'	26
3.5.2. 내장 서블릿 컨테이너 지원	26
3.6. 보안	26
3.6.1. OAuth2	26
3.6.2. 사용자 정보 속 토큰 유형	26
3.6.3. 사용자정보 RestTemplate 재정의	26

3.6.4. 액추에이터 보안(Actuator Security)	26
3.7. SQL 데이터베이스 이용	26
3.7.1. DataSource 구성	26
3.7.2. JdbcTemplate 사용	26
3.7.3. JPA 그리고 '스프링 Data'	26
3.7.4. H2 사용	26
3.8. NoSQL 기술 이용	27
3.8.1. 레디스(Redis)	27
3.8.2. 몽고DB(MongoDB)	27
3.8.3. 솔(Solr)	27
3.8.4. 엘라스틱서치(Elasticsearch)	27
3.9. 캐시	27
3.9.1. 지원하는 캐시 제공자(Provider)	27
3.10. REST 서비스 호출	28
3.10.1. RestTemplate 재정의	28
3.11. 스프링 세션	28
3.12. 테스트	28
3.12.1. 테스트 관련 의존성	28
3.12.2. 스프링 애플리케이션 테스트	28
3.12.3. 스프링 부트 애플리케이션 테스트	28
3.12.4. 테스트 유틸리티	28
3.13. 웹소켓(WebSocket)	29
3.14. 웹 서비스(Web Service)	29
3.15. 자동구성 직접 생성하기	29
3.15.1. 자동구성 빈 이해하기	29
3.15.2. 자동구성 대상자 위치	29
3.15.3. 조건 애너테이션	29
3.15.4. 스타터 직접 생성하기	29
3.16. 이어질 이야기	29
4. 스프링 부트 액추에이터: 출시준비기능	30
4.1. 출시준비 기능 활성화	30
4.1.1. 종단점(Endpoint)	30
4.1.2. 종단점 재정의	30
4.1.3. 액추에이터 MVC 종단점 를 위한 하이퍼미디어	30
4.1.4. CORS 지원	30
4.1.5. 재정의 종단점 추가하기	30
4.1.6. 건강정보	30
4.1.7. HealthIndicator 를 사용한 보안	30
4.1.8. 애플리케이션 정보	30
4.2. HTTP 로 모니터링하고 관리하기	30
4.2.1. 종단점 에 대한 노출강도 보안관리	30
4.2.2. 관리 종단점 경로 재정의	30
4.2.3. 관리 서버 포트 재정의	30
4.2.4. 관리를 위한 SSL 재정의	30
4.2.5. 관리 서버 주소 재정의	31
4.2.6. HTTP 종단점 비활성화	31
4.2.7. HTTP 건강 종단점 접근 제한	31

4.3. 원격셀을 통해 모니터링하고 관리하기	31
4.3.1. 원격셀로 접속	31
4.3.2. 원격셀 확장	31
4.3.3. 원격셀 플러그인	31
4.4. 측정(Metrics)	31
4.4.1. 시스템 측정	31
4.4.2. 데이터소스 측정	31
4.4.3. 캐시 측정	31
4.4.4. 톰캣 세션 측정	31
4.4.5. 측정 기록	31
4.4.6. 공개 측정 추가	31
4.5. 감시	31
4.6. 추적	31
4.7. 프로세스 모니터링	31
4.7.1. 확장 구성	31
4.7.2. 프로그래밍적인 처리	31
4.8. 이어질 이야기	31
5. 스프링 부트 애플리케이션 배포	32
5.1. 클라우드 배포	32
5.1.1. 아마존웹서비스(AWS, Amazon Web Service)	32
5.1.2. 구글앱엔진	32
5.1.3. 클라우드 파운드리(Cloud Foundry)	32
5.2. 스프링 부트 애플리케이션 설치	32
5.2.1. 유닉스/리눅스 서비스	32
5.3. 마이크로소프트 윈도우즈 서비스	32
5.4. 이어질 이야기	32
6. 빌드 도구 플러그인	33
6.1. 스프링 부트 메이븐 플러그인	33
6.1.1. 플러그인 포함	33
6.1.2. 실행가능한 jar 와 war 파일 패키징	33
6.2. 스프링 부트 그레이들 플러그인	33
6.2.1. 플러그인 포함	33
6.2.2. 그레이들 의존성 관리	33
6.2.3. 실행가능한 jar 와 war 파일 생성	33
6.2.4. 스프링 부트 플러그인 구성	33
6.2.5. 리패키징 구성	33
6.2.6. 그레이들 플러그인이 어떻게 동작하는지 이해하기	33
6.2.7. 그레이들 을 이용해서 메이븐 리파지토리에 아티팩트 배포	33
6.3. 다른 빌드 시스템 지원	33
6.3.1. 리패키징 아카이브	33
6.3.2. 내포된 라이브러리	33
6.3.3. 메인 클래스 찾기	33
6.3.4. 리패키지 구현 예제	33
6.4. 이어질 이야기	33
7. 스프링 부트 활용 안내	34
7.1. 스프링 부트 애플리케이션	34
7.1.1. 자동구성 문제해결	34
7.1.2. Environment 혹은 ApplicationContext 가 시작하기 전에 재정의하기	34

7.1.3. ApplicationContext 계층 조직(부모 혹은 루트 컨텍스트 추가)	34
7.1.4. 비-웹애플리케이션 생성	34
7.2. 속성 및 구성	34
7.2.1. 빌드 시 자동으로 속성 확장	34
7.2.2. SpringApplication 외부 구성	34
7.2.3. 애플리케이션 속성 외장파일 위치변경	34
7.2.4. 명령줄 인자 축약사용	34
7.2.5. 외부속성에 대한 YAML 사용	34
7.2.6. 스프링 프로파일 활성화	34
7.2.7. 환경 의존적인 구성 변경	34
7.2.8. 외부속성에 대한 빌트인 옵션(built-in option) 발견	34
7.3. 내장서블릿 컨테이너	34
7.3.1. 애플리케이션에 서블릿, 필터 혹은 리스너 추가하기	34
7.3.2. HTTP 포트 변경	34
7.3.3. 할당되지 않은 HTTP 포트 무작위 사용	34
7.3.4. 실행시 HTTP 포트 발견	35
7.3.5. SSL 구성	35
7.3.6. 접근로그 구성	35
7.3.7. 프론트엔드 프록시 서버 이용	35
7.3.8. 톰캣 구성	35
7.3.9. 톰캣을 통한 다중 커넥터 활성화	35
7.3.10. 톰캣 7.x 혹은 8.0 이용	35
7.3.11. @ServerEndpoint 를 사용한 웹소켓 종단점 생성	35
7.3.12. HTTP 응답 압축 활성화	35
7.4. 스프링 MVC	35
7.4.1. JSON REST 서비스 작성	35
7.4.2. XML REST 서비스 작성	35
7.4.3. 잭슨(Jackson) 오브젝트매퍼(ObjectMapper) 재정의	35
7.4.4. @ResponseBody 랜더링 재정의	35
7.4.5. Multipart 파일 업로드 제어	35
7.4.6. 스프링 MVC DispatcherServlet 전환	35
7.4.7. 기본 MVC 구성 전환	35
7.4.8. ViewResolver 재정의	35
7.5. HTTP 클라이언트	35
7.5.1. RestTemplate 가 프록시를 사용하도록 구성	35
7.6. 로깅	35
7.6.1. Logback 구성	36
7.6.2. Log4j 구성	36
7.7. 데이터 접근	36
7.7.1. DataSource 구성	36
7.7.2. DataSource 복수 정의	36
7.7.3. 스프링 Data 리파지토리 이용	36
7.7.4. 스프링 구성으로부터 @Entity 정의 분리	36
7.7.5. JPA 속성 구성	36
7.7.6. EntityManagerFactory 재정의 이용	36
7.7.7. 복수 EntityManager 이용	36
7.7.8. 전통적인 persistence.xml 이용	36

7.7.9. 스프링 Data JPA 와 몽고 리파지토리 이용	36
7.7.10. REST 종단점 로 스프링 Data 리파지토리 노출	36
7.8. 데이터베이스 초기화	36
7.8.1. JPA 를 이용한 데이터베이스 초기화	36
7.8.2. Hibernate 를 이용한 데이터베이스 초기화	36
7.8.3. 스프링 JDBC 를 이용한 데이터베이스 초기화	36
7.8.4. 스프링 배치 를 이용한 데이터베이스 초기화	36
7.8.5. 고차원 데이터베이스 마이그레이션 도구 사용	36
7.9. 배치 애플리케이션	37
7.9.1. 구동시 스프링 배치 작업 실행	37
7.10. 액추에이터	37
7.10.1. 액추에이터 종단점 의 포트 혹은 주소 변경	37
7.10.2. whitelabel 오류 페이지 재정의	37
7.10.3. 액추에이터 과 Jersey	37
7.11. 보안	37
7.11.1. 스프링 부트 보안 구성 전환	37
7.11.2. AuthenticationManager 변경하고 사용자 계정 추가	37
7.11.3. 프록시 서버 뒤에서 실행될 때 HTTPS 활성화	37
7.12. 핫스와핑(Hot swapping)	37
7.12.1. 정적 콘텐츠 재적재	37
7.12.2. 컨테이너 재시작없이 템플릿 재적재	37
7.12.3. 빠른 애플리케이션 재시작	37
7.13. 빌드	37
7.13.1. 빌드 정보 생성	37
7.13.2. 깃 정보 생성	37
7.13.3. 의존성 버전 재정의	37
7.13.4. Java 6 에서 실행하기	37
7.14. 전통적 배포	37
7.14.1. 배포가능한 war 파일 생성	37
7.14.2. 오래된 서블릿 컨테이너에 배포가능한 war 파일 생성	38
7.14.3. 기존 애플리케이션을 스프링 부트 로 변환	38
7.14.4. 오래된 컨테이너(Servlet 2.5)에 war 배포	38
8. 정리	39
부록	40
Appendix A: 개발환경 설정	41
JDK설치	41
STS 설치	41
그레이들 설치	41
Appendix B: 개발 시 참고사항	42
스프링부트 레퍼런스 문서 참고	42
관련 자동구성 클래스 확인	42
debug=true 를 이용한 활성화된 자동설정과 비활성화된 자동설정 확인	42
실행가능한 jar 구조	42
공통 애플리케이션 속성들	42
스택오버플로에서 스프링부트 관련 질문 찾기	42
Appendix C: 참고문헌	43
Spring Framework	43
Spring Boot	43

Spring Security 43

Spring Test 43

저자 소개

안녕하세요, 평범한 일반인 **김지현** 입니다.

인터넷에서는 허니몬(**honeymon**) 이라는 이름으로 활동하고 있습니다.

항상 자기소개를 할 때면 수줍습니다.

'일반인'을 자처하면서 '개발자'가 읽을 책을 써보겠다고 나서는 무모함을 가지고 있는 평범한 사람입니다. 스쿠버다이빙, 클라이밍, 라이딩, 수영 등의 취미생활을 즐기는 것 만큼 개발자로서 일해오는 것을 즐겁게 생각하고 있습니다. 생물학과 컴퓨터과학을 복수전공하면서 어느쪽으로 갈까 고민하다가 잠시 전산쪽으로 빠져 있었습니다. 그러다가 그쪽의 일이 들어지면서 진로를 고민하던 차에 사촌 덕분에 이 바닥에 뛰어들게 되었습니다. 년수로는 8년차가 되었네요.

빠르게 발전하고 변화해가는 IT기술 속에서 '자바' 그 중에서 스프링 을 기반으로 한 '웹 애플리케이션' 을 개발하게 된 건 어찌보면 당연하다 여겨질만큼 우리나라 시장에서 많은 수요가 있습니다. 대부분은 SI(System Integration) 라고 하는 특수한 환경이 비정상적으로 비대해지면서 생겨난 특이한 상황이죠. 저는 그런 상황 속에서 꽤 운이 좋은 편이었습니다. 5개월의 '정부지원 자바 전문가 과정'을 마치고 공공기관의 유지보수(SM, System Maintenance) 에 투입되어 1년 동안 생활하면서 '이건 나랑 안맞아' 라고 생각하고 회사에 배치변경을 요구했지만 묵살당하면서 그에 대한 반감을 가지면서 였습니다. 때마침 '이일민'님께서 '토비의 스프링 3.0'을 출간하면서 한국스프링유저그룹(KSUG, Korea Spring User Group)에서 책임기 모임이 있었습니다. 그 책임기 모임을 시작으로 KSUG 와의 길고긴 인연이 지금까지 이어져서 현재는 KSUG 일꾼단에서 3년 넘게 활동을 하고 있습니다. 그 덕분에 이런저런 경험을 하면서 많은 사람들을 만나면서 조금은 개발자 흉내를 낼 수 있게 된 듯 합니다.

제가 스프링 부트 와 인연을 맺게 된 것은, 잘 다니던 회사를 2014년 5월 말에 그만두고 '스쿠버 다이빙 강사' 가 되겠다며 양양 동해바다에서 3개월 동안 생활하다가 복귀하면서 스테디에서 알던 형과 함께 솔루션 개발에 스프링 부트 를 적용하기로 한 것이 계기가 되었습니다.

다양한 운영체제에도(JVM 기반) 단독 실행이 가능한(Executable jar) 웹애플리케이션(Spring MVC)으로 개발해야한다.

는 요구사항에 제일 먼저 떠오른 것이 '스프링 부트' 였습니다. 그 무렵에 스프링 부트와 관련된 소개하는 발표가 몇 번 있었습니다. 그래도 귀동냥한 것이 있어서 써먹을 수 있게 되었습니다. 스프링 부트로 개발을 하면서 제일 많이 보게되는 것은 역시 [참고문서\(refernece document\)](#) 였습니다. 스프링 은 문서화가 정말 잘 되어 있습니다. 스프링 부트를 보는 사람이 많이 늘어나겠다는 생각에 오픈소스이고 거기에 문서화도 [아스키독\(AsciiDoc\)](#) 으로 작성되어 있었기에 '레퍼런스 문서' 번역을 해보기로 결심했습니다. 이 번역을 하기에 앞서 에이콘에서 '[Git을 이용한 버전관리](#)' 라는 책을 번역했었기에 쉽게 할 수 있을 거라고 봤습니다(... 부끄럽고 죄송하다).

무식하면 용감하다

라던가? [Spring Boot Reference Guide](#) 를 그대로 복사하여 번역할 계획을 세웠습니다. 이제는 요령이 생겼지만, 처음 시작할 때는 'current' 와 'current-snap' 과 특정 버전의 문서를 각각 볼 수 있다는 것과, 스프링 부트 가 빠르게 버전업된다는 것을 살피지 못했습니다. 그래도 용감하게 번역을 시작합니다.

- [스프링 부트 참고가이드](#)

저장소에 들어오면 바로 볼 수 있도록 하겠다는 생각으로 **README.md** 파일에 정말 무식하게 했다. 그러다가 지쳐서 딴짓하다가 다른 분들에게 도움을 받아 80% 정도의 번역을 끝으로 방치상태로 내버려둡니다(... 마무리를 지어볼까 하는 시점에는 1.3.0 이 나와버렸고 제대로 구성하지 않은 문서를 가지고는 변경사항들만 적용하기도 어려워져 포기하고 1.4.0

을 제대로 해볼까?). 이 무모했던 번역을 마무리 짓고 있던 제게 또 다른 번역 제안이 들어왔습니다. 내용도 괜찮고 분량도 적당하다는 생각에 그리고 5개월 동안 번역하고 손질하는데 많은 시간이 들고서 'Spring MVC 4 익히기' 라는 책이 나왔습니다. 이 책을 번역하고 있는 동안에 몇 곳에서 스프링 부트 관련 번역서 제안과 집필 제안이 있었다.

그리고 현재 이 책을 쓰고 있습니다. 두둑치…..!

들어가며

스프링 부트가 [소개\(2014.04.01\)](#)된 지 2년이 넘은 현재 시점에 들어서 스프링 부트에 대한 책들이 나오기 시작했다. 각각의 책들은 각자의 방식으로 스프링 부트를 소개하고 있으며, 이 책을 쓰고 있는 나 역시 나만의 방식으로 소개하면서 책을 읽는 여러분이 각자의 방식으로 소화하여 자유롭게 스프링 부트 를 사용할 수 있도록 유혹하려고 한다. [스프링 부트 참고가이드](#) 를 번역했던 경험을 바탕으로 스프링 부트로 개발하면서 정말 '스프링 부트 참고 문서'를 많이 봤다. 거기서 나아가 소스코드를 열어보면서 어떻게 구성되어 있고 동작할지를 살피기 시작했고 그런 이해를 바탕으로 필요한 경우 구성을 변경하거나 제공하는 클래스와 인터페이스를 사용하여 직접 구성하는 경험을 쌓게 되었다. 그런 경험을 전달하여 삽질(or 시행착오)을 줄이면서 빠르게 적응하길 바라는 마음으로 이 책을 쓴다.

이 책은 스프링 부트 참고문서를 기반으로 하여 그에 대응하는 설명과 '삽질경험' 을 기술한다.

책에서 다루는 내용

[시작하기](#)에서는, 스프링 부트 에 대해서 간단하게 소개하면서 스프링 부트 의 기본적인 개발흐름을 눈에 익힌다. 스프링 부트는, 스프링 기반의 웹 애플리케이션을 개발하던 때와는 확연하게 다른 개발경험을 개발자들에게 선사한다. 이는 다른 언어가 가지고 있던 장점들을 자바 개발자들에게 부여하며 '프로토타이핑'-'개발'-'운영' 까지 이어지도록 해주는 근사한 경험을 선물한다. 거기에는 XML 기반의 문맥(Context) 으로 애플리케이션을 정의하는 흐름이 자바코드로 작성하며 구성(Configuration) 하는 개념으로 변모하는 역동적인 변화가 발생한다.

[첫 스프링 부트 애플리케이션 살펴보기](#)에서는 스프링 부트 를 본격적으로 들여다본다. 기능구현에 필요한 다른 라이브러리들을 손쉽게 추가할 수 있는 '의존성 관리', 추가된 '의존성' 라이브러리들을 바로 사용할 수 있도록 스프링 부트 개발팀이 권장하는 '자동구성(Auto-Configuration)', 개발 생산성을 높여주고 확장성을 부여하는 '스프링 프레임워크'와의 조합, 내장컨테이너를 이용한 빠른 실행과 재가동 시간을 줄이는 '개발자 도구' 등 스프링 부트 를 이용하는데 알아둬야할 것들을 살펴본다.

[스프링 부트 기능](#)에서는 스프링 부트 를 자유자재로 응용할 수 있는 근간이 되는 스프링 부트의 기능들을 살펴본다. 스프링 부트 의 진입점(Entry point)가 되는 `SpringApplication` 을 시작으로 해서 배포를 위해 구성된 배포파일을 다시 빌드하지 않아도 실행환경에 따라 설정값을 변경할 수 있으며, 프로파일을 통해서 '로컬','단위테스트','개발','운영' 등의 독립적인 실행환경별로 구성하고, 다양한 생태계와 연동하고, 필요에 따라 추가적인 구성을 작성하는 과정 등을 살펴보게 된다. 이 장을 읽으면서 [스프링 부트](#)의 매력에 흠뻑 취하게 될 것이다.

[스프링 부트 액추에이터: 출시준비기능](#)에서는 '빠른 개발' 이후에 고민하게 되는 '운영' 의 묘미를 살려줄 수 있는 다양한 기능을 살펴본다. 개발하여 배포하고 운영되고 있는 스프링 부트 를 모니터링하고 그 정보를 수집하고 경우에 따라 쉽게 제어할 수 있는 기능들을 소개한다.

[스프링 부트 애플리케이션 배포](#)에서는 스프링 부트 로 개발된 웹 애플리케이션을 배포하는 방법을 살펴본다. 편리하고 막강한 기능을 제공하는 클라우드 서비스들이 많이 등장했다. 그 클라우드 서비스에 우리가 만든 앱을 배포하는 과정을 살펴 보면서 자신에게 맞는 클라우드 서비스를 선택하고 관리해보자.

[빌드 도구 플러그인](#)에서는 '실행가능한(Executable)' 배포본(jar or war)를 만들어내는 '빌드 도구' 를 살펴보도록 한다. 스프링 부트 는 자바 개발시 주로 사용하는 빌드도구인 메이븐과 그레이들 에서 사용할 수 있는 빌드 플러그인을 제공하고 있다. 이 빌드 플러그인을 이용하여 빌드할 때 고려할 수 있는 다양한 선택사항들을 제공하여 개발자의 다양한 욕구를 수용하고 있다. 여기서 '실행가능한' 배포본을 만들어내는 교묘한 기술 '재포장(리패키징, Repackage)' 에 대해 설명한다.

[스프링 부트 활용 안내](#)에서는 스프링 부트 로 개발하는 과정에서 고려해볼 다양한 확장 포인트들을 살펴본다. 스프링 부트 는 기본적으로 개발팀에서 권장(Recommend) 하는 방식이 있다. 그렇지만 스프링 부트 를 사용하는 개발자에게 다양한

확장 포인트를 제공하고 있다. 이는 스프링 부트 의 근간이 되는 스프링 프레임워크가 가지고 있는 특징이기도 하다. '다 써볼 수 있을까?' 싶을만큼 너무나 다양하고 방대한 내용을 다루고 있다.

정리 은 앞에서 다뤘던 내용들을 정리한다. 미리 결론부터 이야기하자면 '자바 개발자+스프링 프레임워크 사용자' 라면 **스프링 부트** 를 능숙하게 쓰면 좋다는 것이다.

책을 따라하기 위해 필요항목

스프링 부트 의 **시스템 요구사항** 을 보면 2.0.0.RELEASE 은 **Java 7** 과 Spring Framework 4.3.5.RELEASE 혹은 상위버전을 요구한다. Java 6도 지원을 하지만 사용하는 **컨테이너의 종류**에 따라서 제한되거나 영향을 받을 수 있다. 책에서 작성한 코드들 중에 일부는 Java 8에서 제공하는 **Optional**, **stream()** 등을 사용하기에 Java 8을 설치하고 작성하길 바란다.

IDE 는 **STS** 와 **IntelliJ** 등이 있는데 이 책에서는 STS 를 기준으로 한다. 스프링 부트에 대한 기능 지원이 괜찮기도 하고 이클립스 기반이라 많은 개발자들에게 익숙하고 무료로 사용할 수 있다는 강점(?)이 있어 부담없이 사용할 수 있다.

대상독자

스프링 부트를 기반으로 한 개발에 관심을 가지고 이제 막 예제를 만들어보려 하거나 스프링 프레임워크에 대한 기본적인 지식이 있는 개발자를 대상으로 한다. 스프링 부트를 제대로 활용하기 위해서는 스프링 프레임워크와 그에 기반되는 지식을 가지고 있는 것이 좋다. 스프링 부트는 'spring + spring-boot + auto-configuration + build-tool + 3rd party' 의 조합이기 때문에 자바 프로그래밍을 처음 시작하는 이들에게는 조금은 부담스러울 수 있다. 스프링 부트를 제대로 사용하려면 스프링에 대한 공부도 병행해야 한다.

관례(Convention)

책을 읽어가다보면 반복적으로 사용되는 표현들이 있을 것이다. **코드**는 다음과 같이 작성된다.

```
public class Honeymon {
    private String firstName;
    private String lastName;
    private Gender gender;
    private Integer age;
    private String site;
    private Integer weight;

    public Fat eat(Food food) {
        this.weight += food.getWeight();
    }
}
```

코드 중에서 중요한 부분에 대해서는 다음과 같이 코드 오른쪽에 숫자가 매겨지고 하단에 그 숫자에 설명이 기술된다.

```
public class Honeymon {
    private String firstName;
    private String lastName;
    private Gender gender;
    private Integer age;
    private String site;
    private Integer weight;

    public Fat eat(Food food) {
        this.weight += food.getWeight(); ①
    }
}
```

① 먹은 만큼 찐다! 만고불변의 법칙!

중요한 단어나 문장에 대해서는 **굵은 글씨**로 처리한다. **스프링 부트**는 프레임워크다. 문장 중간에 노출되는 클래스명과 같은 코드성 표현에 대해서는 **Honeymon**으로 표현한다.

중요해서 기억해두었으면 하는 부분에 대해서는 다음과 같이 **노트**로 표기한다.

NOTE 노트

알아두면 유용한 것에 대해서는 다음과 같이 **유용함**으로 표기한다.

TIP 유용함!

인용한 것에 대해서는 **인용**으로 표기한다.

인용

용어나 개념을 소개하는 경우에는 한국어로 표현하는 것을 선호하는 편이다. 첫 표기시에는 **자동구성(Configuration)**과 **설정(Set-Up)**과 같이 한국어와 영어를 병행표기하고 이후 표기에서는 한국어만 표기한다. 적절한 한국어 표현을 찾기 어려운 경우에는 **액추에이터(Actuator)**처럼 외래어 표기법을 따른다.

이 표기에 대해서는 의견이 분분하다. 어떤 이들은 영어를 그대로 표기하는 게 좋다하는 사람도 있지만, 나는 억지스러울지도 모르겠지만 한국어로 표기하고자 하는 노력을 기울이고 있다. 그렇다고 해서 나만 이해할 수 있는 엉터리 표기는 피하려고 노력한다. 이에 대해서는 각자의 방식으로 접근하고 가장 효율적인 방법을 선택하는 것이 적합하다고 본다.

예제

이 책에서 사용된 예제는 <https://github.com/ihoneymon/boot-spring-boot>에서 내려받을 수 있다. 메이븐이나 그레이들 없이 실행할 수 있도록 래퍼(wrapper)가 구성되어 있지만 이 래퍼를 실행하기 위해서는 JVM이 있어야 한다.

프로젝트는 JDK8 을 기준으로 작성되었다. 필요하다면 리파지토리를 복제(clone) 해서 JDK7 에서도 실행가능하게 구성하고 공유해주면 다른 개발자들에게 도움이 될 것이다.

Chapter 1. 시작하기

스프링 부트를 소개하면서 스프링 부트를 이용한 기본적인 개발흐름을 눈에 익혀본다. 스프링 부트는 개발자들에게 그 이전에 스프링 기반의 웹 애플리케이션을 개발하던 때와는 확연하게 다른 개발경험을 선사해 주었다. 이는 다른 언어가 가지고 있던 '빠른 프로토타이핑' 및 '개발/운영' 까지 이어지도록 해주는 근사한 경험을 자바 개발자들에게도 제공했다. 그 안에는 개발자가 애플리케이션에 필요한 구조를 작성하는 문맥(**Context**) 이라고 하는 흐름이 애플리케이션에 필요한 컴포넌트와 설정을 조합하는 구성(**Configuration**) 이라는 개념으로 변모하는 역동적인 변화가 있었다.

스프링 부트가 제공하는 신세계로 함께 항해를 떠나자.

1.1. 스프링 부트 소개

스프링 부트는 '개발 플랫폼'이다. 스프링 프레임워크를 기반으로, 관례적인 자동구성을 제공하고 자바 개발환경에서 사용되는 기능들을 쉽게 추가하고, '프로토타이핑'-'개발'-'운영/모니터링' 까지 이어지는 다양한 기능을 지원하고 있다.

스프링 부트 프로젝트 페이지(<http://projects.spring.io/spring-boot>) 스프링 부트 기능 정의를 살펴보면 스프링 부트의 특징을 이해할 수 있다.

- 단독실행 가능한 스프링 애플리케이션 생성
- 내장 컨테이너로 톰캣, 제티 혹은 언더투우 중에서 선택가능(WAR 배포 필요없음)
- 스타터 'starter' POM 을 제공하여 간결한 의존성 구성 지원
- 스프링에 대한 {auto-configuration} 제공
- 제품출시 후 운영에 필요한 상태, 건강 점검과 외부구성 등 다양한 기능 제공
- 구성을 위한 XML 코드를 필요로 하지 않으며 작성하지 않아도 됨

위의 여섯가지 기능 정의를 통해 스프링 부트를 어떻게 사용해볼 수 있을지 궁리해볼 수 있을 것이다.

1.1.1. 단독실행 가능한 스프링 애플리케이션 생성

순수한 형태의 자바 애플리케이션을 작성하고 WAS 에 배포하는 별도의 작업없이도 내장 컨테이너를 이용하여 애플리케이션을 실행할 수 있다. 이 과정은 스프링 애플리케이션 생성 및 프로토타이핑을 매우 용이하게 해준다.

1.1.2. 내장 컨테이너로 톰캣, 제티 혹은 언더투우 중에서 선택가능(WAR 배포 필요없음)

스프링 부트의 **spring-boot-starter-web** 는 기본적으로는 톰캣을 포함하고 있지만, 개발자가 원한다면 언제든지 톰캣을 제거하고 제티 혹은 언더투우를 내장 컨테이너로 대체할 수 있다.

1.1.3. 스타터 'starter' POM 을 제공하여 간결한 의존성 구성 지원

메이븐 을 이용한 의존성 선언은 관련되거나 필요한 기타 의존성을 함께 선언할 수 있다. 스프링 부트 프로젝트에 있는 **pom.xml** 혹은 **build.gradle** 을 열었을 때 볼 수 있는 의존성(**dependency**) 부분에 **spring-boot-starter-*** 들은 각각의 컴포넌트가 실행되는데 필요한 의존성들을 내포하고 있다. 이에 대해서는 **스타터** 에서 다루도록 하겠다.

1.1.4. 스프링에 대한 {auto-configuration} 제공

`@EnableAutoConfiguration` 애너테이션 을 이용해서 각 컴포넌트별 기본적인 자동구성 을 지원한다. 이에 대해서는 [자동구성\(Auto-configuration\)](#) 에서 다루도록 하겠다.

1.1.5. 제품출시 후 운영에 필요한 상태, 건강 점검과 외부구성 등 다양한 기능 제공

개발을 마친 애플리케이션을 배포한 후에는 애플리케이션에서 발생하는 오류 뿐만 아니라 애플리케이션이 동작하면서 사용하는 자원의 상태 및 살아있는지 여부를 확인할 수 있는 방법이 필요하다. 이런 기능을 별도로 개발하지 않아도 될만큼 다양한 기능을 제공해준다. 이를 활용하는 방법에 대해서는 [스프링 부트 액츄에이터: 출시준비기능](#) 에서 상세히 다룬다.

1.1.6. 구성을 위한 XML 코드를 필요로 하지 않으며 작성하지 않아도 됨

스프링 부트에서 가장 큰 변화 중 하나는 더이상 **XML 구성**을 필요로 하지 않는다는 것이다. 자바구성(`JavaConfig`) 을 통해서 애플리케이션에 필요한 컴포넌트 스캔(`ComponentScan`) 과 스프링 빈(`Bean`) 등록 등을 할 수 있게 되었다. 이마저도 관례적인 자동구성(`Auto-Configuration`)을 제공하여 애플리케이션 구성에 대한 부담감을 줄여준다. 스프링 부트를 기반으로 개발하는 개발자는 환경구성에 대한 부담없이 비즈니스 로직구현에만 집중할 수 있도록 해준다. 물론 `@Configuration` 가 선언된 클래스에서 `@ImportResource` 애너테이션을 이용해서 패키지에 포함된 **XML** 파일을 읽어올 수 있는 기능을 제공한다. 이에 대해서는 [XML 구성 불러오기](#) 에서 살펴본다.

스프링 부트 의 여섯가지 기능적 특징을 간단하게 살펴봤다.

1.2. 시스템 요구사항

스프링 부트2.0.0.RELEASE는 기본적으로 [Java 7](#) 과 Spring Framework 4.3.5.RELEASE혹은 상위버전을 요구한다. 스프링 부트버전에 따라 적재된 스프링프레임워크의 버전은 다르니 이를 확인하기 바란다. 스프링 부트가 정식출시되면서 스프링4.X 버전을 적재하고 있기에 이에 맞춰 **애플리케이션의 기능을 설계하고 구현하기를 권한다**. 필요하다면 Java 버전과 스프링버전을 변경할 수는 있지만 그에 따라 별도로 구성을 작성해야 하는 것들이 있고 최근에 나온 플러그인들이나 라이브러리들이 Java 7에 맞춰진 경우가 많아서 특별한 경우가 아니라면 Java 7을 기준으로 사용하길 권한다.

NOTE

Java 6 를 사용하는 방법에 대해서는 <http://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/#howto-use-java-6> 를 살펴보자.

NOTE

스프링 부트개발팀에서는 가능하면 **Java 8 을 사용하기**를 권한다. 스프링프레임워크 4.x 부터 Java 8 을 완벽하게 지원했다. 나 역시, Java 8 에서 제공하는 스트림(Stream) API 와 랴다식의 혜택을 누릴 수 있도록 Java 8 을 사용하기를 권한다.

- [Java 8 새로운 점](#)

애플리케이션에 필요한 기능들을 가진 의존성을 확보하기 위해서는 빌드도구를 사용하는 것이 편리하다. 빌드도구는 애플리케이션 패키징 외에도 의존성 관리, 빌드전처리 등의 작업을 지원한다. 스프링 부트는 자바 개발환경의 대표적인 빌드도구인 메이븐과 그레이들에서 사용할 수 있는 플러그인을 제공한다.

이 빌드도구를 통해서 메이븐 저장소에서 starter POM 을 가져오고 그와 관련된 라이브러리들을 추가할 수 있다.

NOTE

이 책에서는 그레이들을 기본 빌드도구로 이야기를 진행한다. 그레이들래퍼(wrapper)를 이용하게 되면 다른 팀원들은 프로젝트 빌드를 위해서 그레이들을 설치하지 않아도 된다. 이에 대한 설명은 [빌드 시스템](#)에서 자세히 할 예정이다.

TIP

메이븐도 래퍼가 존재한다. 사용하는 방식은 유사하니 넘어가도록 하자. 이에 대한 정보는 [Maven Wrapper](#)와 [Maven Wrapper 소개와 사용 - 윤병화, pop-it](#)에서 볼 수 있다.

Java 사용버전을 고려하는 데 있어서 가장 큰 영향을 주는 것은 사용하는 컨테이너가 무엇이나 하는 것이다. 스프링 부트는 기본적으로 톰캣(tomcat)을 내장형 컨테이너로 사용하지만 필요에 따라 손쉽게 제티(jetty)와 언더투우(Undertow)로 변경이 용이하다.

1. 내장 서블릿 컨테이너별 지원사양

Name	Servlet Version	Java Version
Tomcat 8	3.1	Java 7+
Tomcat 7	3.0	Java 6+
Jetty 9.3	3.1	Java 8+
Jetty 9.2	3.1	Java 7+
Jetty 8	3.0	Java 6+
Undertow 1.3	3.1	Java 7+

NOTE

이 외에도 스프링 부트는 서블릿 3.0+ 이상을 지원하는 컨테이너라면 어디에든 배포가능하다.

- 참고: [Servlet container](#)

NOTE

웹소켓 등 컨테이너에 따라 의존성을 가지고 있는 라이브러리 등이 영향을 받기 때문에 컨테이너를 선정할 때는 이런 영향도를 고려하기 바란다. 간단한 예로 웹소켓([JSR-356](#)) 구현체가 각기 다르기에 컨테이너 변경시 영향을 많이 받을 수 있다.

- [Tomcat WebSocket\(org.apache.tomcat.websocket\)](#)
- [Jetty WebSocket\('org.eclipse.jetty.websocket'\)](#)

1.3. 스프링 부트 설치

스프링 부트 를 이용하기 위해서 설치해야할 것은 없다.

스프링 부트가 나온 이후 대부분의 IDE에서는 스프링 부트와 관련된 편의기능을 제공한다. STS의 경우에는 스프링 부트용 대시보드를 제공하여 시작, 디버깅, 원격지 배포실행 등의 기능을 제공하고 있다. 많은 책에서 스프링 부트 CommandLine Client 설치, 실행방법과 프로젝트 생성 방식을 다루지만 여기서는 다루지 않는다.

NOTE

개인적으로 스프링 부트 CommandLine Client 를 사용한 경험이 거의 없다. IDE에서 'Spring Initializer'를 이용해서 프로젝트를 생성하고 관리해왔다. 터미널 사용을 좋아한다면 한번 익혀보는 것도 좋을 듯 하다. 익혀볼 의사가 있다면 [Installing the Spring Boot CLI \(https://goo.gl/d9wkiw\)](https://goo.gl/d9wkiw)를 살펴보기 바란다.

빌드도구의 래퍼 기능을 이용하면 별도로 빌드도구를 설치할 필요가 없다. 이에 대한 내용은 [래퍼](#) 를 살펴보기 바란다.

1.4. 스프링 부트 애플리케이션 개발 시작

스프링 부트 애플리케이션을 개발하기 위해서는 플러그인과 의존성을 포함한 프로젝트를 생성해야 한다.

1.4.1. 프로젝트 생성

STS(Spring Tool Suite, <https://spring.io/tools/sts>) 와 IntelliJ(<https://www.jetbrains.com/idea/>) 에서 프로젝트를 생성한다. 이후에는 STS 를 기준으로 진행한다.

IDE에서 스프링 부트 프로젝트를 생성하는 방식은 'Spring Initializer' 사이트에서 생성한 **zip** 파일을 내려받아 이를 지정된 위치에 압축해제한 후에 불러내기(import) 한다.

NOTE

'Spring Initializer' 에서 프로젝트 초기파일을 내려받는 방식이기 때문에 인터넷 연결이 되지 않는 경우에는 스프링 부트프로젝트를 생성하기 어렵다. 이를 해결하기 위한 방법으로 인트라넷에 'spring initializer' 프로젝트를 가동해야할 듯 하다. 여기에 오픈소스의 위대함이 깃드는데, [Spring Initializr \(https://github.com/spring-io/initializr/\)](https://github.com/spring-io/initializr/) 프로젝트에서 소스를 내려받아 인트라넷에 구성할 수 있다.

STS(or Eclipse): 스프링 부트 프로젝트 생성

1. 'File' - 'New' - 'Spring Starter Project' 선택
image::[Spring Starter Project 생성하기]
2. 프로젝트 정보를 다음과 같이 입력하고 '[Next >]' 버튼 클릭 image::[프로젝트 정보 입력]
 - 'Type' 선택
 - 'Maven'
 - 'Gradle(Buildship)'
 - 'Gradle(STS)' ← 선택
 - 'Packaging' 선택
 - 'Jar': WAS 에 배포하지 않고 커맨드로 실행시 선택
 - 'War': WAS 에 배포하는 경우 선택
 - 'Java' 선택
 - '1.8' ← 권장
 - '1.7'

- '1.6'
- 'Language' 선택
 - 'Java'
 - 'Groovy'
 - 'Kotlin'
- group: `io.honeymon`
- artifact: `boot-spring-boot`
- package: `io.honeymon.springboot.book`

3. 스프링 부트 버전 및 의존성을 선택하고 '[Finish]' 를 누른다. image::[선택항목]

- 선택항목: `web`, `jpa`, `h2`, `lombok`, `thymeleaf`
 - `web`
 - `jpa`
 - `h2`
 - `lombok`
 - `thymeleaf`
- '[Make Default]' 버튼을 누르면 'Frequently Used' 항목에 추가된다.
- '[Next >]'를 누르는 경우 다음과 같은 정보를 확인할 수 있다. image::[Spring Boot initializer Site]

```
http://start.spring.io/starter.zip?name=boot-
demo&groupId=io.honeymon&artifactId=bootdemo&version=0.0.1-
SNAPSHOT&description=Demo+project+for+Spring+Boot&packageName=io.ho
neymon.bootdemo&type=gradle-
project&packaging=jar&javaVersion=1.8&language=java&bootVersion=1.4
.4.RELEASE&dependencies=lombok&dependencies=data-
jpa&dependencies=h2&dependencies=thymeleaf&dependencies=web
```

해당 창에 있는 'Full URL' 항목을 브라우저 주소창에 복사해서 붙여넣으면 zip 파일이 'artifact' 에 입력값을 파일명으로 가지는 `bootdemo.zip` 와 같은 파일이 다운로드 된다. 이 파일을 특정위치로 옮겨 압축해제하고 IDE에서 불러오는 것도 가능하다.

IntelliJ: 스프링 부트 프로젝트 생성

1. 'File' → 'New' 선택
2. 'Spring Initializer' 선택
 - a. SDK 선택

b. [Next] 선택

3. 'New Project' 창에서 프로젝트 기본정보 등록 → [Next]
4. 사용될 "의존성" 선택 → [Next]
5. 프로젝트 설치 위치 지정 → [Finish]

1.4.2. 의존성 추가

`build.gradle` 중에서 의존성에 대한 선언이 된 부분이다.

```
dependencies {  
    compile('org.springframework.boot:spring-boot-starter-web')  
  
    testCompile('org.springframework.boot:spring-boot-starter-test')  
}
```

`build.gradle` 중에서 `dependencies` 안에 의존성을 선언한다. [Chapter 7. Dependency Management Basics \(https://goo.gl/iLaf7v\)](https://goo.gl/iLaf7v) 를 살펴보면 `compile`, `testCompile` 등의 정보를 확인할 수 있다.

1.4.3. 예제: `ExampleController` 작성

[코드 구조](#) 에서 설명하긴 하겠지만, 애플리케이션 메인 클래스를 제외한 새롭게 생성하는 클래스들은 기본 `default` 패키지에 두지 않기를 권한다. 지금 작성하는 코드는 어디까지나 간단하게 코드작성법을 설명하기 위한 예제일 뿐이다.

앞에서 보여준 과정을 따라 했다면 `BootSpringBootApplication` 혹은 변경한 프로젝트 명을 접두사로 가지는 `~Application` 클래스가 자동생성되어 있다.

NOTE

`BootSpringBootApplication` 클래스는 수정할 일이 별로 없다. 여기에 {spring-boo} 의 마술이 있다. 이 마술에 대해서는 [@SpringBootApplication 애노테이션 이용](#) 에서 설명한다.

이 클래스가 위치한 클래스에서 `api` 패키지를 생성 한다. 그 후 `ExampleController` 이란 이름으로 클래스를 생성하자. 이 클래스는 이후에 여러분이 자주 작성하게될 API 클래스의 기본구조를 가진다.

```

package io.honeymon.springboot.book.api;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController ①
public class ExampleController {

    @GetMapping("/greeting") ②
    public String helloWorld() {
        return "Hello, World";
    }
}

```

① `ExampleController` 클래스에서 제일 먼저 선언된 `@RestController` 은 스프링 4.0 때 추가된 메타 애너테이션 이다. `@Controller` 와 `@ResponseBody` 애너테이션 을 포함하고 있다.

② 스프링 4.3 에 추가된 메타 애너테이션 으로 `@RequestMapping(method = RequestMethod.GET)` 을 짧게 쓸 수 있다. '그렇다면? 혹시...' 하고 생각하는데 {enum} `RequestMethod` 에 선언되어 있는 `GET`, `HEAD`, `POST`, `PUT`, `PATCH`, `DELETE`, `OPTIONS`, `TRACE` 에 대응하는 `~Mapping` 들이 존재한다.

1.4.4. 예제: `ExampleController` 실행하기

이제 예제를 실행해보자. `BootSpringBootApplication` 를 우클릭하고 팝업메뉴에서 'Run as' → 'Spring Boot App' 을 선택한다.

'Spring Boot Dashboard' 의 항목이 변경된다. 콘솔(Console) 창에 스프링부트가 실행되는 과정에서 로그가 출력된다.

`BootSpringBootApplication` 실행모습

image::[]

브라우저에서 <http://localhost:8080> 으로 접근해보면 다음과 같이 `Hello, World` 가 출력되는 것을 볼 수 있다.

애플리케이션을 중지시키려면 콘솔이나 IDE 의 정지버튼을 누르면 된다.

1.4.5. 실행가능한 JAR 만들기

예제의 마지막 단계로 내장 컨테이너(Embedded Container) 를 포함한, 실제로 운영에서도 사용가능한 형태의, 실행가능한 JAR 를 생성해보자. 실행가능한 JAR('Fat Jars'라고도 불림)는 우리가 작성한 코드를 컴파일한 클래스와 이를 실행하는데 필요한 모든 의존성 jar 가 함께 압축되어 있다(여기에는 톰캣과 같은 WAS도 내장형 컨테이너로 포함되는데 여기에는 스프링 부트 플러그인의 `bootRepackage` 가 관여한다).

실행가능한 JAR 와 Java

Java does not provide any standard way to load nested jar files (i.e. jar files that are themselves contained within a jar). This can be problematic if you are looking to distribute a self-contained application.

To solve this problem, many developers use “uber” jars. An uber jar simply packages all classes, from all jars, into a single archive. The problem with this approach is that it becomes hard to see which libraries you are actually using in your application. It can also be problematic if the same filename is used (but with different content) in multiple jars.

Spring Boot takes a different approach and allows you to actually nest jars directly.

실행가능한 JAR를 생성하기 위해서는 `build.gradle` 에 `spring-boot-gradle-plugin` 가 추가되어야 한다. `build.gradle` 첫번째 줄에 다음 코드를 추가하자.

```
buildscript {  
    ext {  
        springBootVersion = '2.0.0.RELEASE'  
    }  
    repositories {  
        mavenCentral()  
    }  
    dependencies {  
        classpath("org.springframework.boot:spring-boot-gradle-plugin:  
${springBootVersion}")  
    }  
}
```

① `buildscript` 선언을 통해서 그레이들 빌드시 `spring-boot-gradle-plugin` 을 이용할 수 있게 된다.

NOTE

스프링 부트 플러그인을 설치하면 `bootRepackage`, `bootRun` 태스크(task)를 실행할 수 있게 된다. 이 태스크들에 대해서는 [스프링 부트 그레이들 플러그인](#) 에서 설명한다. 그레이들의 `buildscript` 에 대한 설명은 [Using your plugin in another project\(https://goo.gl/vdE77Q\)](https://goo.gl/vdE77Q) 을 읽어보길 바란다.

프로젝트를 선택하고 우클릭하여 'Run AS' → 'Grdle' → 'Task Quick Launcher' 를 클릭한다. 팝업된 실행창에서 `bootRepackage` 를 입력하자. 자동완성 기능을 제공하기 때문에 `boot` 까지 입력하면 `bootRun` 과 `bootRepackage` 가 나열될 것이다.

image::[Gradle Task Quick Launcher

`bootRepackage]`

```

[sts] -----
[sts] Starting Gradle build for the following tasks:
[sts]      bootRepackage
[sts] -----
:compileJava
:processResources UP-TO-DATE
:classes
:findMainClass
:jar
:bootRepackage

BUILD SUCCESSFUL

Total time: 1.31 secs
[sts] -----
[sts] Build finished succesfully!
[sts] Time taken: 0 min, 1 sec
[sts] -----

```

위와 같은 내용이 콘솔에 출력될 것이다. `boot-spring-boot` 프로젝트에서 `build/libs` 폴더를 보면 `boot-spring-boot-0.0.1-SNAPSHOT.jar` 와 `boot-spring-boot-0.0.1-SNAPSHOT.jar.original` 이 생성되어 있다. `boot-spring-boot-0.0.1-SNAPSHOT.jar.original` 는 우리가 작성한 코드가 컴파일되어 있다. 이를 스프링 부트 플러그인에서 실행에 필요한 컨테이너와 의존성을 추가하여 재압축(repackage) 한 것이 `boot-spring-boot-0.0.1-SNAPSHOT.jar` 으로 대략 10MB 정도의 용량이 늘어나게 된다. 이에 대해서는 [빌드 도구 플러그인](#) 에서 자세히 설명한다.

`jar tvf` 명령으로 `boot-spring-boot-0.0.1-SNAPSHOT.jar` 내에 어떤 것들이 담겼는지 확인해볼 수 있다.

```
$ jar tvf build/libs/boot-spring-boot-0.0.1-SNAPSHOT.jar
```

`bootRepackage` 을 통해서 생성된 `boot-spring-boot-0.0.1-SNAPSHOT.jar` 파일은 `java -jar` 명령으로 커맨드라인에서 실행할 수 있다.

```
$ java -jar build/libs/boot-spring-boot-0.0.1-SNAPSHOT.jar
... 스프링부트 로고
:: Spring Boot ::                (v1.5.1.RELEASE)

... i.h.s.book.BootSpringBootApplication : Starting
BootSpringBootApplication on jake-mac.local with PID 42838
(/Users/honeymon/workspace/boot-spring-boot/build/libs/boot-spring-boot-
0.0.1-SNAPSHOT.jar started by honeymon in /Users/honeymon/workspace/boot-
spring-boot)
... i.h.s.book.BootSpringBootApplication : No active profile set,
falling back to default profiles: default
... 로그 생략 ..
... [main] s.b.c.e.t.TomcatEmbeddedServletContainer : Tomcat
started on port(s): 8080 (http)
... [main] i.h.s.book.BootSpringBootApplication : Started
BootSpringBootApplication in 2.668 seconds (JVM running for 3.09)
```

실행중인 애플리케이션은 **Ctrl + C** 를 누르면 간단하게 종료된다.

1.5. 이어질 이야기

스프링 부트가 동작하는 기본 구성과 원리를 설명한다.

Chapter 2. 첫 스프링 부트 애플리케이션 살펴보기

기능을 구현하는데 필요한 다른 라이브러리들을 손쉽게 추가할 수 있는 '의존성 관리', 추가된 '의존성' 라이브러리들을 바로 사용할 수 있도록 스프링 부트 개발팀이 권장하는 '자동구성(Auto-Configuration)', 개발 생산성을 높여주고 확장성을 부여하는 '스프링 프레임워크'와의 조합, 내장컨테이너를 이용한 빠른 실행과 재가동 시간을 줄이는 '개발자 도구' 등 스프링 부트 를 이용하는데 알아둬야할 것들을 살펴본다.

2.1. 빌드 시스템

2.1.1. 의존성 관리

2.1.2. 스타터

2.1.3. 메이븐

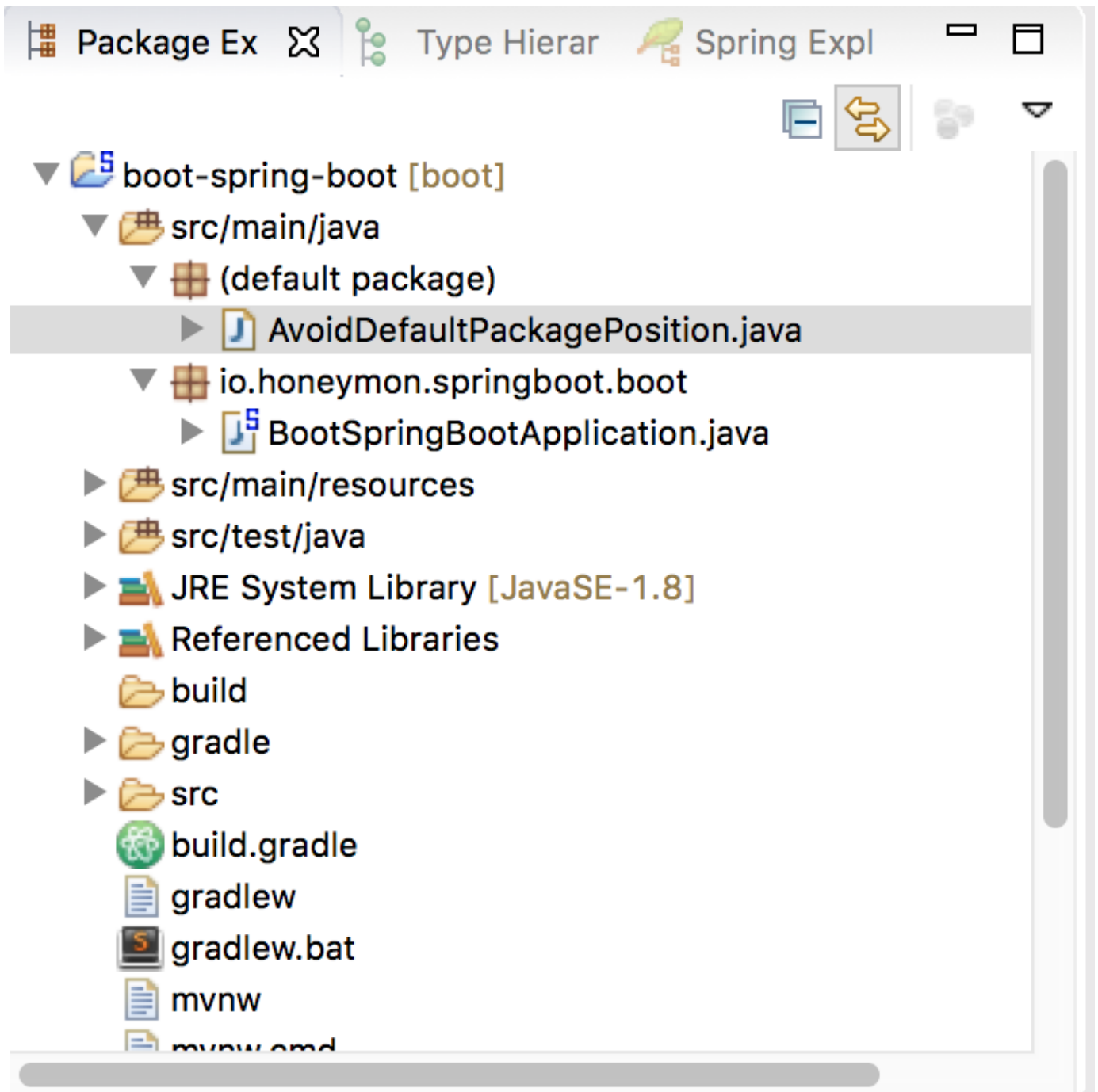
2.1.4. 그레이들

2.1.5. 래퍼

2.2. 코드 구조

2.2.1. "기본 패키지" 이용

기본패키지("Default package") 로 간주되는 되는 곳에 클래스를 작성하는 것은 좋지 않다. 기본 패키지 사용은 권장되지 않으니 피하는 것이 좋다.



스프링부트에서 `@ComponentScan`, `@EntityScan` 혹은 `@SpringBootApplication` 애너테이션이 모든 jar 에서 모든 클래스를 탐색하는 상황이 발생할 수도 있다.

TIP

자바에서 권장하는 패키지명 작성관례를 따르거나 도메인명을 뒤집어서 사용하는 것을 권장한다(예: `io.honeymon.springboot`).

고전적으로 사용된 방식은 도메인명을 뒤집는 것이기는 하지만 길어지는 것이 싫다면, 개발팀 내의 합의가 있다면 축약해서 사용해도 된다. 특별히 형식이 정해져 있는 것은 아니니 자신만의 방식을 사용하여 작성해보자(예: `io.honeymon.springboot` → `io.honey.sb`).

2.2.2. 애플리케이션 메인 클래스 위치

최상위 패키지 에 메인클래스가 있고 `@SpringBootApplication` 가 있는 경우

```
io
+- honeymon
  +- bsb
    +- BootSpringBootApplication.java
    |
    +- domain
    |   +- Customer.java
    |   +- CustomerRepository.java
    |
    +- service
    |   +- CustomerService.java
    |
    +- web
    |   +- CustomerController.java
```

`BootSpringBootApplication` 는 다음처럼 간단해진다.

```
@SpringBootApplication
public class BootSpringBootApplication {

    public static void main(String[] args) {
        SpringApplication.run(BootSpringBootApplication.class, args);
    }
}
```

그런데 `BootSpringBootApplication` 의 위치가 `app` 디렉토리로 변경된다면,

메인클래스 **app** 으로 이동한 경우

```
io
+- honeymon
  +- springboot.book
    +- app
    |   +- BootSpringBootApplication.java
    |
    +- domain
    |   +- Customer.java
    |   +- CustomerRepository.java
    |
    +- service
    |   +- CustomerService.java
    |
    +- web
    |   +- CustomerController.java
```

BootSpringBootApplication 에 **scanBasePackages** 속성에 탐색해야할 패키지를 지정해야하는 번거로움이 생긴다. 의외로 **@SpringBootApplication** 이 선언된 메인 클래스 위치를 임의적으로 변경하고 애플리케이션 이 실행되지 않는다(혹은 에러가 발생한다)는 질문을 올리는 이들이 꽤 많다.

```
@SpringBootApplication(scanBasePackages = {
    "io.honeymon.springboot.book.domain", "
    io.honeymon.springboot.book.service", "io.honeymon.springboot.book.web" })
@EnableJpaRepositories("io.honeymon.springboot.book.domain")
@EntityScan("io.honeymon.springboot.book.domain")
public class BootSpringBootApplication {

    public static void main(String[] args) {
        SpringApplication.run(BootSpringBootApplication.class, args);
    }
}
```

위처럼 선언하지 않으면, Controller 를 찾지 못하거나 빈을 찾지 못하여 예외가 발생한다.

```
Caused by:
org.springframework.beans.factory.NoSuchBeanDefinitionException: No
qualifying bean of type
[io.honeymon.springboot.book.domain.CustomerRepository] found for
dependency [io.honeymon.springboot.book.domain.CustomerRepository]:
expected at least 1 bean which qualifies as autowire candidate for this
dependency. Dependency annotations:
{@org.springframework.beans.factory.annotation.Autowired(required=true)}
at
org.springframework.beans.factory.support.DefaultListableBeanFactory.raise
NoMatchingBeanFound(DefaultListableBeanFactory.java:1406)
// 생략
```

스프링 애플리케이션 구성에 어느 정도 익숙해지면 개인적인 역량에 따라 메인 클래스 위치를 변경할 수 있겠지만, 최상위 패키지 에 메인 클래스를 위치시키는 게 여러모로 편리하다는 것을 알아두길 바란다. 프로젝트의 패키지 구성은 사소한 듯 하면서도 신경쓸 부분들이 많은데, 이와 관련해서는 [패키지 구조는 어떻게 가져가는 것이 좋을까?](#), [Get Your Java Package Structures Right](#) 와 [Package by feature, not layer](#) 등을 읽어본 후에 팀원들과 이야기를 나누며 적절한 구조를 잡길 바란다. 딱 '이게 정답이다! 이렇게 해라!' 라고 할 수 있는 건 없다.

2.3. 구성(Configuration) 클래스

2.3.1. 추가적인 구성클래스 불러오기

2.3.2. XML 구성 불러오기

2.4. 자동구성(Auto-configuration)

스프링 부트 개발팀에서 범용적으로 사용되는 구성을 JavaConfig 와 ~.properties 를 구성하여 사용하려는 기능의 라이브러리가 추가되면 조건에 따라 자동적으로 선택되는 구성클래스에 대한 이야기

2.4.1. 자동구성 대체하기

미리 마련되어 있는 인터페이스와 추상 클래스를 이용하여 이용자가 자신에게 맞춰 재정의하여 사용한다.

2.4.2. 특정 자동구성 비활성화하기

사전 계획에 따라 의존성은 추가해줬지만 당장은 필요없는 경우에 자동구성을 꺼둔다.

2.5. 스프링 빈과 의존성 주입

2.6. @SpringBootApplication 애노테이션 이용

2.7. 애플리케이션 실행하기

2.7.1. IDE 에서 실행하기

2.7.2. 패키징된 애플리케이션 실행하기

2.7.3. 메이븐 플러그인으로 실행하기

2.7.4. 그레이들 플러그인으로 실행하기

2.7.5. 핫 스와핑

2.8. 스프링 부트 개발자도구

2.8.1. 기본 속성들

2.8.2. 자동 재시작

2.8.3. 전역 설정

2.8.4. 원격 애플리케이션

2.9. 출시를 위한 애플리케이션 패키징

2.10. 이어질 이야기

Chapter 3. 스프링 부트 기능

스프링 부트의 진입점(Entry point) `SpringApplication`을 시작으로, 패키징된 애플리케이션을 배포환경에 따라 별도로 빌드할 필요없이 배포환경에 따라 각각의 속성을 부여하는 방법, 프로파일을 통해서 '로컬','테스트','개발','운영' 등의 실행환경을 구분하고, 다양한 서드파티 생태계와 연동하고, 필요에 따라 구성(Configuration)을 작성하는 과정 등을 살펴보게 된다. 이 장을 읽으면서 스프링 부트의 매력에 흠뻑 취하게 될 것이다.

3.1. 스프링애플리케이션(SpringApplication)

3.1.1. 배너 재정의

3.1.2. `SpringApplication` 재정의

3.1.3. 유연한(Fuent) 빌더 API

3.1.4. 애플리케이션 이벤트 및 리스너

3.1.5. 웹 환경

3.1.6. 애플리케이션 실행 인자에 접근

3.1.7. `ApplicationRunner` 혹은 `CommandLineRunner` 사용하기

3.1.8. 애플리케이션 종료

3.1.9. 관리자 기능

3.2. 외부 구성

3.2.1. 랜덤값 구성

3.2.2. 명령줄 속성 접근

3.2.3. 애플리케이션 속성 파일

3.2.4. 프로파일 정의 속성

3.2.5. 속성 내 치환자(placeholder)

3.2.6. `properties` 를 대신하여 `YAML` 사용하기

YAML 적재

{spring-environment} 에서 `properties` 처럼 `YAML` 사용하기

YAML 문서로 다중 프로파일 다루기

YAML 단점

YAML 목록 병합(Merging)

3.2.7. 보장형 구성 속성

서드파티 구성

느슨한 연결

`Properties` 관례

`@ConfigurationProperties` 유효성검사

`@ConfigurationProperties` 대 `@Value`

3.3. 프로파일

3.3.1. 활성화할 프로파일 추가하기

3.3.2. 프로파일을 프로그래밍적으로 설정

3.3.3. 프로파일 정의 구성 파일

3.4. 로깅

3.4.1. 로그 형식

3.4.2. 콘솔 출력

3.4.3. 파일 출력

3.4.4. 로그 레벨

3.4.5. 로그 구성 재정의

3.4.6. 로그백 확장

3.5. 웹애플리케이션 개발

3.5.1. 'Spring Web MVC framework'

3.5.2. 내장 서블릿 컨테이너 지원

서블릿, 필터 와 리스너

서블릿 컨텍스트 초기화

`EmbeddedWebApplicationContext`

내장 서블릿 컨테이너 재정의

JSP 제한

3.6. 보안

3.6.1. OAuth2

공인(`Authorization`) 서버

자원(`Resource`) 서버

3.6.2. 사용자 정보 속 토큰 유형

3.6.3. 사용자정보 `RestTemplate` 재정의

클라이언트

`Single Sign On`

3.6.4. 액츄에이터 보안(`Actuator Security`)

3.7. SQL 데이터베이스 이용

3.7.1. `DataSource` 구성

3.7.2. `JdbcTemplate` 사용

3.7.3. JPA 그리고 '스프링 Data'

3.7.4. H2 사용

프로파일 구성

H2 웹콘솔 사용

3.8. NoSQL 기술 이용

3.8.1. 레디스(Redis)

3.8.2. 몽고DB(MongoDB)

몽고DB 데이터베이스 연결

MongoTemplate

스프링 Data 몽고DB 리파지토리

내장 몽고

3.8.3. 솔(Solr)

솔 연결하기

스프링 Data 솔 리파지토리

3.8.4. 엘라스틱서치(Elasticsearch)

스프링 Data 를 통해 엘라스틱서치 연결

스프링 Data 엘라스틱서치

3.9. 캐시

3.9.1. 지원하는 캐시 제공자(Provider)

Generic

JCache

EhCache 2.x

Hazelcast

Redis

Guava

Simple

None

3.10. REST 서비스 호출

3.10.1. `RestTemplate` 재정의

3.11. 스프링 세션

3.12. 테스트

3.12.1. 테스트 관련 의존성

3.12.2. 스프링 애플리케이션 테스트

3.12.3. 스프링 부트 애플리케이션 테스트

테스트 구성 탐색

테스트 구성 제외

랜덤포트 이용

빈에 대한 목(Mock) 과 스파이(Spy) 처리

자동구성된 테스트

자동구성된 JSON 테스트

자동구성된 스프링 MVC 테스트

자동구성된 Data JPA 테스트

자동구성된 REST 클라이언트

자동구성된 스프링 REST Docs 테스트

스폭(Spock) 을 사용하여 스프링 부트 애플리케이션 테스트

3.12.4. 테스트 유틸리티

`ConfigFileApplicationContextInitializer`

`EnvironmentTestUtils`

`OutputCapture`

3.13. 웹소켓(WebSocket)

3.14. 웹 서비스(Web Service)

3.15. 자동구성 직접 생성하기

3.15.1. 자동구성 빈 이해하기

3.15.2. 자동구성 대상자 위치

3.15.3. 조건 애너테이션

클래스 조건

빈 조건

속성 조건

자원 조건

웹애플리케이션 조건

SpEL 표현식 조건

3.15.4. 스타터 직접 생성하기

이름짓기

자동구성 모듈

스타터 모듈

3.16. 이어질 이야기

Chapter 4. 스프링 부트 액츄에이터: 출시준비기능

개발한 애플리케이션을 배포하고, 배포실행되고 있는 스프링 부트 애플리케이션을 모니터링하고 그 정보를 수집하고 경우에 따라 제어할 수 있는 기능들을 소개한다.

4.1. 출시준비 기능 활성화

4.1.1. 종단점(Endpoint)

4.1.2. 종단점 재정의

4.1.3. 액츄에이터 MVC 종단점 를 위한 하이퍼미디어

4.1.4. CORS 지원

4.1.5. 재정의 종단점 추가하기

4.1.6. 건강정보

4.1.7. `HealthIndicator` 를 사용한 보안

4.1.8. 애플리케이션 정보

자동구성된 `InfoContributors`

재정의한 애플리케이션 정보

깃(Git) 커밋 정보

빌드(build) 정보

`InfoContributors` 재정의 작성

4.2. HTTP 로 모니터링하고 관리하기

4.2.1. 종단점 에 대한 노출강도 보안관리

4.2.2. 관리 종단점 경로 재정의

4.2.3. 관리 서버 포트 재정의

4.2.4. 관리를 위한 SSL 재정의

4.2.5. 관리 서버 주소 재정의

4.2.6. HTTP 종단점 비활성화

4.2.7. HTTP 건강 종단점 접근 제한

4.3. 원격셀을 통해 모니터링하고 관리하기

4.3.1. 원격셀로 접속

4.3.2. 원격셀 확장

4.3.3. 원격셀 플러그인

4.4. 측정(Metrics)

4.4.1. 시스템 측정

4.4.2. 데이터소스 측정

4.4.3. 캐시 측정

4.4.4. 톰캣 세션 측정

4.4.5. 측정 기록

4.4.6. 공개 측정 추가

4.5. 감시

4.6. 추적

4.7. 프로세스 모니터링

4.7.1. 확장 구성

4.7.2. 프로그래밍적인 처리

4.8. 이어질 이야기

개발한 애플리케이션을 모니터링하고 관리하는 방법을 살펴봤으니, 실제로 다양한 시스템에 배포해보자. 이 과정이 반복되면 웹 애플리케이션을 배포하고 운영하는 노하우가 쌓이게 될 것이고 다양한 응용할 수 있게 될 것이다.

Chapter 5. 스프링 부트 애플리케이션 배포

편리하고 막강한 기능을 제공하는 클라우드 서비스들이 많이 등장했다. 그 클라우드 서비스에 우리가 만든 앱을 배포하는 과정을 살펴보면서 자신에게 맞는 클라우드 서비스를 선택하고 관리해보자.

5.1. 클라우드 배포

5.1.1. 아마존웹서비스(AWS, Amazon Web Service)

5.1.2. 구글앱엔진

5.1.3. 클라우드 파운드리(Cloud Foundry)

5.2. 스프링 부트 애플리케이션 설치

5.2.1. 유닉스/리눅스 서비스

5.3. 마이크로소프트 윈도우즈 서비스

5.4. 이어질 이야기

Chapter 6. 빌드 도구 플러그인

에서는 '실행가능한(Executable)' 배포본(jar or war)를 만들어내는 '빌드 도구' 를 살펴보도록 한다. 스프링 부트는 자바 개발시 주로 사용하는 빌드도구인 메이븐과 그레이들 에서 사용할 수 있는 빌드 플러그인을 제공하고 있다. 이 빌드 플러그인을 이용하여 빌드할 때 고려할 수 있는 다양한 선택사항들을 제공하여 개발자의 다양한 욕구를 수용하고 있다. 여기서 '실행가능한' 배포본을 만들어내는 교묘한 기술 '재포장(리패키징, Repackage)' 에 대해 설명한다.

6.1. 스프링 부트 메이븐 플러그인

6.1.1. 플러그인 포함

6.1.2. 실행가능한 jar 와 war 파일 패키징

6.2. 스프링 부트 그레이들 플러그인

6.2.1. 플러그인 포함

6.2.2. 그레이들 의존성 관리

6.2.3. 실행가능한 jar 와 war 파일 생성

6.2.4. 스프링 부트 플러그인 구성

6.2.5. 리패키징 구성

6.2.6. 그레이들 플러그인이 어떻게 동작하는지 이해하기

6.2.7. 그레이들 을 이용해서 메이븐 리파지토리에 아티팩트 배포

6.3. 다른 빌드 시스템 지원

6.3.1. 리패키징 아카이브

6.3.2. 내포된 라이브러리

6.3.3. 메인 클래스 찾기

6.3.4. 리패키지 구현 예제

6.4. 이어질 이야기

Chapter 7. 스프링 부트 활용 안내

스프링 부트는 기본적으로 개발팀에서 권장(Recommend) 하는 방식이 있다. 그렇지만 스프링 부트를 사용하는 개발자에게 다양한 확장 포인트를 제공하고 있다. 이는 스프링 부트의 근간이 되는 스프링 프레임워크가 가지고 있는 특징이기도 하다. '다 써볼 수 있을까?' 싶을만큼 너무나 다양하고 방대한 내용을 다루고 있다.

7.1. 스프링 부트 애플리케이션

7.1.1. 자동구성 문제해결

7.1.2. `Environment` 혹은 `ApplicationContext` 가 시작하기 전에 재정의하기

7.1.3. `ApplicationContext` 계층 조직(부모 혹은 루트 컨텍스트 추가)

7.1.4. 비-웹애플리케이션 생성

7.2. 속성 및 구성

7.2.1. 빌드 시 자동으로 속성 확장

7.2.2. `SpringApplication` 외부 구성

7.2.3. 애플리케이션 속성 외장파일 위치변경

7.2.4. 명령줄 인자 축약사용

7.2.5. 외부속성에 대한 YAML 사용

7.2.6. 스프링 프로파일 활성화

7.2.7. 환경 의존적인 구성 변경

7.2.8. 외부속성에 대한 빌트인 옵션(built-in option) 발견

7.3. 내장서블릿 컨테이너

7.3.1. 애플리케이션에 서블릿, 필터 혹은 리스너 추가하기

7.3.2. HTTP 포트 변경

7.3.3. 할당되지 않은 HTTP 포트 무작위 사용

7.3.4. 실행시 HTTP 포트 발견

7.3.5. SSL 구성

7.3.6. 접근로그 구성

7.3.7. 프론트엔드 프록시 서버 이용

7.3.8. 톰캣 구성

7.3.9. 톰캣을 통한 다중 커넥터 활성화

7.3.10. 톰캣 7.x 혹은 8.0 이용

7.3.11. `@ServerEndpoint` 를 사용한 웹소켓 종단점 생성

7.3.12. HTTP 응답 압축 활성화

7.4. 스프링 MVC

7.4.1. JSON REST 서비스 작성

7.4.2. XML REST 서비스 작성

7.4.3. 잭슨(Jackson) 오브젝트매퍼(`ObjectMapper`) 재정의

7.4.4. `@ResponseBody` 렌더링 재정의

7.4.5. Multipart 파일 업로드 제어

7.4.6. 스프링 MVC `DispatcherServlet` 전환

7.4.7. 기본 MVC 구성 전환

7.4.8. `ViewResolver` 재정의

7.5. HTTP 클라이언트

7.5.1. `RestTemplate` 가 프록시를 사용하도록 구성

7.6. 로깅

7.6.1. Logback 구성

7.6.2. Log4j 구성

7.7. 데이터 접근

7.7.1. DataSource 구성

7.7.2. DataSource 복수 정의

7.7.3. 스프링 Data 리파지토리 이용

7.7.4. 스프링 구성으로부터 @Entity 정의 분리

7.7.5. JPA 속성 구성

7.7.6. EntityManagerFactory 재정의 이용

7.7.7. 복수 EntityManager 이용

7.7.8. 전통적인 persistence.xml 이용

7.7.9. 스프링 Data JPA 와 몽고 리파지토리 이용

7.7.10. REST 종단점 로 스프링 Data 리파지토리 노출

7.8. 데이터베이스 초기화

7.8.1. JPA 를 이용한 데이터베이스 초기화

7.8.2. Hibernate 를 이용한 데이터베이스 초기화

7.8.3. 스프링 JDBC 를 이용한 데이터베이스 초기화

7.8.4. 스프링 배치 를 이용한 데이터베이스 초기화

7.8.5. 고차원 데이터베이스 마이그레이션 도구 사용

Flyway

Liquibase

7.9. 배치 애플리케이션

7.9.1. 구동시 스프링 배치 작업 실행

7.10. 액츄에이터

7.10.1. 액츄에이터 종단점 의 포트 혹은 주소 변경

7.10.2. `whitelabel` 오류 페이지 재정의

7.10.3. 액츄에이터 과 Jersey

7.11. 보안

7.11.1. 스프링 부트 보안 구성 전환

7.11.2. `AuthenticationManager` 변경하고 사용자 계정 추가

7.11.3. 프록시 서버 뒤에서 실행될 때 HTTPS 활성화

7.12. 핫스와핑(Hot swapping)

7.12.1. 정적 콘텐츠 재적재

7.12.2. 컨테이너 재시작없이 템플릿 재적재

7.12.3. 빠른 애플리케이션 재시작

7.13. 빌드

7.13.1. 빌드 정보 생성

7.13.2. 깃 정보 생성

7.13.3. 의존성 버전 재정의

7.13.4. Java 6 에서 실행하기

7.14. 전통적 배포

7.14.1. 배포가능한 war 파일 생성

7.14.2. 오래된 서블릿 컨테이너에 배포가능한 war 파일 생성

7.14.3. 기존 애플리케이션을 스프링 부트 로 변환

7.14.4. 오래된 컨테이너(Servlet 2.5)에 war 배포

Chapter 8. 정리

스프링 부트 플랫폼을 함께 살펴봤다. 나의 이야기가 마음에 들었는지 모르겠다.

'자바 개발자+스프링 프레임워크 사용자' 라면 **스프링 부트** 를 능숙하게 다룰 수 있다면 쉽고 빠르게 애플리케이션을 개발하고 배포-운영할 수 있다.

부록

Appendix A: 개발환경 설정

JDK설치

STS 설치

그레이들 설치

Appendix B: 개발 시 참고사항

스프링부트 레퍼런스 문서 참고

관련 자동구성 클래스 확인

`debug=true` 를 이용한 활성화된 자동설정과 비활성화된 자동설정 확인

실행가능한 jar 구조

공통 애플리케이션 속성들

스택오버플로에서 스프링부트 관련 질문 찾기

Appendix C: 참고문헌

Spring Framework

Spring Boot

Spring Security

Spring Test